This example is maybe not the most dynamic sketch we've ever written, but it gives us a sense of the types of characters that can be displayed on our display. Once we have the sketch uploaded, we can then look at the functions that we use to both display these symbols and characters as well as those that create the new characters, in order to better explain the example source code.

## write()

Where the `print()` function prints strings of text to the LCD, the `write()` function is used to send a character to the display. The syntax for both functions is similar:

```
write(data)
```

The character to be sent to the display is specified in a numerical data format or through a variable name. From our example code, it is as follows:

```
lcd.write(symbols[i]);
```

This line displays whatever symbol is named at the current index in the `symbols[]` array determined by the loop counter. The `symbols[]` array is just a way to run down a list of each of the eight symbols defined in the earlier variable declarations, using a `for` loop to print them to the display. Our example code continues to print each of the eight pre-defined symbols using the `setCursor()` function to space each one out on the first line.

Declaring the address for the symbol at the beginning of our code and calling that variable name later is only one way to use the `write()` function. We can actually specify the symbol to be displayed in many different ways. For example, the symbol for cents, ¢, can be written multiple ways, as shown in the following sample fragment:

```
byte cents = B11101100;
lcd.write(cents);
lcd.write((B11101100));
lcd.write(236);
lcd.write(0xEC);
```

This block of code will display four ¢ symbols in a row on the display because each of these different values refers to the same thing. Just like our example code earlier, the first line declares a byte data type variable named `cents` and assigns it the 8-bit binary value `B11101100`. The variable `cents` is then called in the second line to print the first symbol. The second symbol is printed by specifying the binary number directly in the `write()` function. Because the value `B11101100` is not a real number so to speak, the `B` is used to tell the compiler that the following number is binary not decimal, we need to place this value in a second pair of parentheses to force the compiler to evaluate this value as a binary number. The third line uses the decimal value 236 and the fourth the hexadecimal value 0xEC for the third and fourth symbols.

To find the numerical value that corresponds to the symbol that we want to use, we need to have a look at the LCD's data sheet, a document that gives us the technical specification of the device. Figure 9-3 is a small part of the symbol set that is available to a particular LCD found in a data sheet for the driver chip, like the one hosted by SparkFun at `www.sparkfun.com/datasheets/LCD/HD44780.pdf`. These may vary depending on the chip used by the LCD, so refer to the specific data sheet for your display.